Noregs miljø- og biovitskaplege universitet

Institutt for datavitskap

Digitalisering på Ås

INF205
Ressurseffektiv programmering

3 Datastrukturar

3.1 Grafdatastrukturar

3.2 Kode-data-samutvikling

3.3 Listedatastrukturar

Noregs miljø- og biovitskaplege universitet

Institutt for datavitskap

Digitalisering på Ås

3 Datastrukturar

3.1 Grafdatastrukturar

3.2 Kode-data-samutvikling

3.3 Lineære datastrukturar

INF205

5. mars 2026

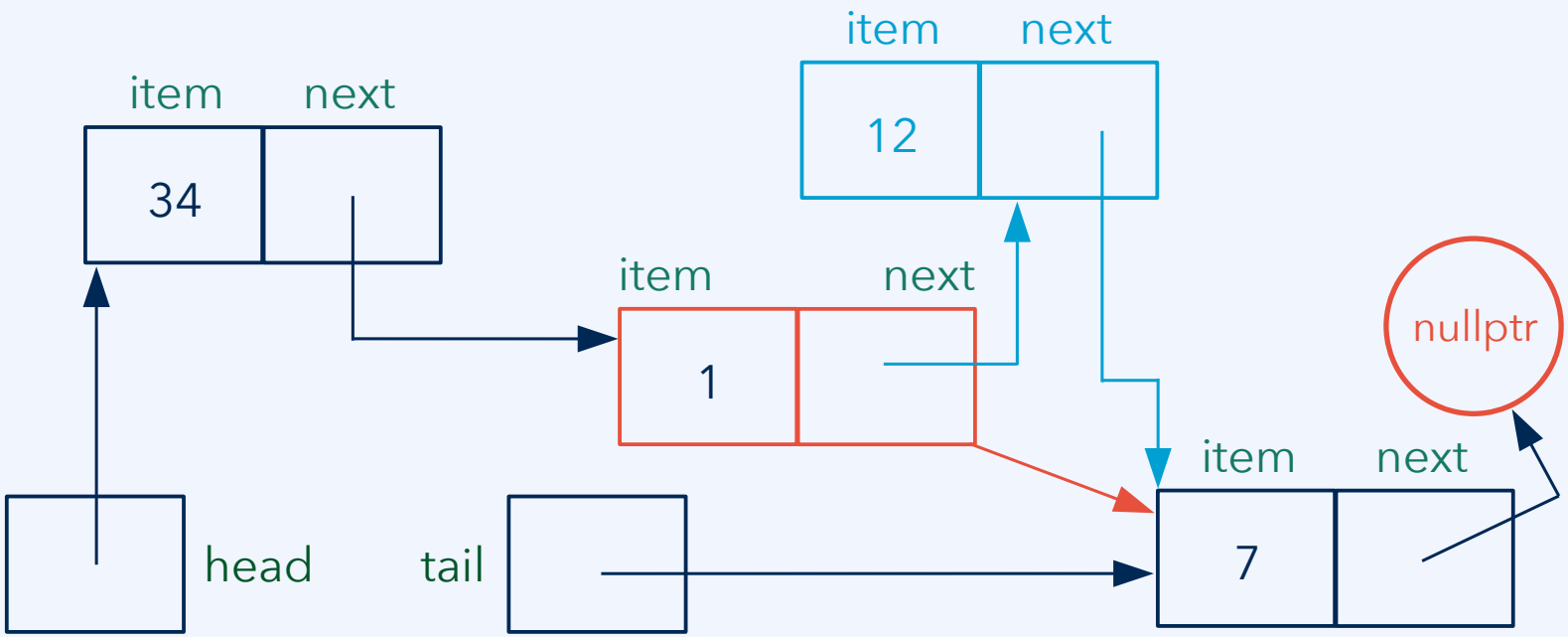
Ordliste

breadth-first search (BFS)	breidd-fyrst-søk (BFS)
complexity class	kompleksitetsklasse
depth-first search (DFS)	djupn-fyrst-søk (DFS)
dynamic data structure	dynamisk datastruktur
queue	kø
stack [data structure]	stakk [datastruktur]

Linked lists (singly linked)

Linked lists are **dynamic data structures**. Their elements are **not contiguous in memory**. Therefore, pointer arithmetics and increments (p++) cannot be used. Instead, the linked list consists of **nodes**.

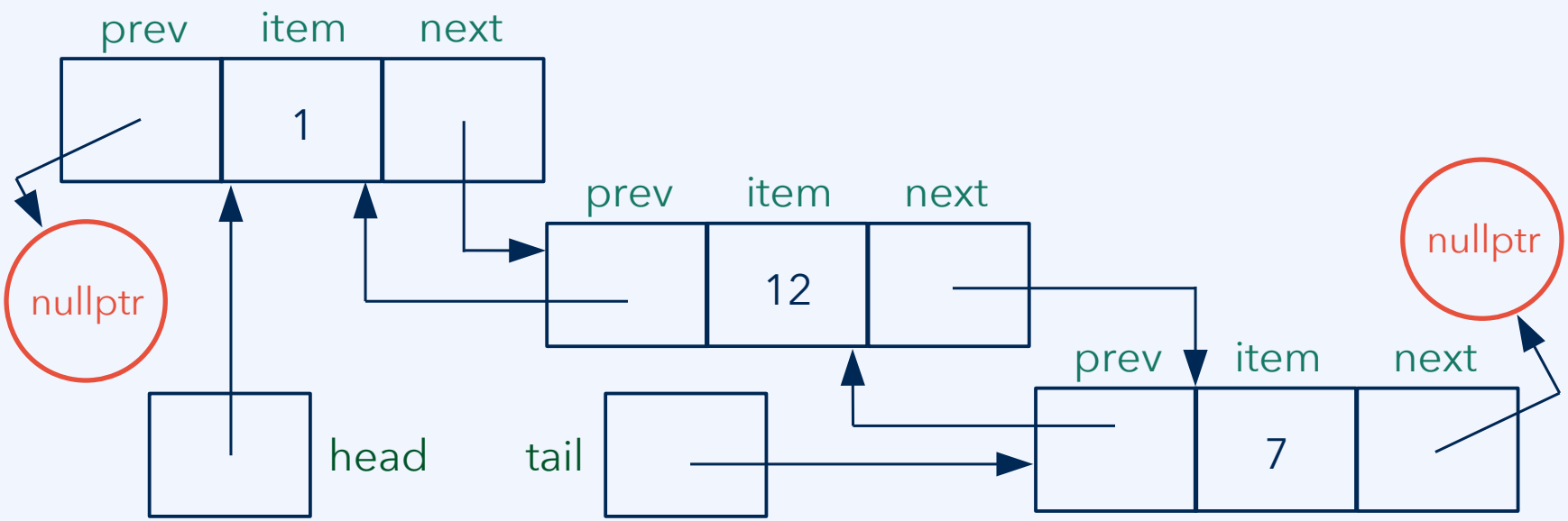
Example task: Insert 12 after node x, to which we already have a reference.



Linked lists (doubly linked)

In a **doubly linked list**, each node also contains a reference (or pointer) to the **previous node**. This facilitates traversal in **both directions and inserting** a new data item **before** any given node (rather than only after it), all in constant time.

Singly linked lists require two variables per data item (item and next).
Doubly linked lists require three variables per data item (prev, item, and next).

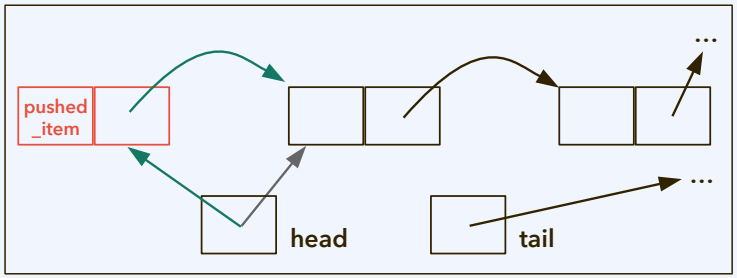


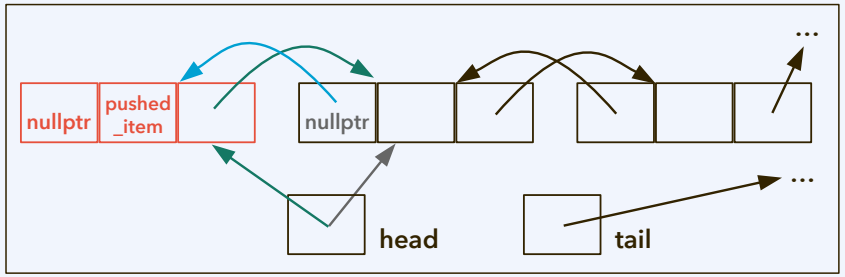
From singly to doubly linked

```
// add an item at the beginning of the list
void SinglyLinkedList::push_front(
  const int& pushed_item
){
  SinglyLinkedListNode* new_node
                  = new SinglyLinkedListNode;
  new_node->set_item(pushed_item);

  if(this->empty()) this->tail = new_node;
  else
    new_node->set_next(this->head);

  this->head = new_node;
}
```

For every link forward (next), there is now also a link backward (prev).

```
// add an item at the beginning of the list
void DoublyLinkedList::push_front(
  const int& pushed_item
){
  DoublyLinkedListNode* new_node
                  = new DoublyLinkedListNode;
  new_node->set_item(pushed_item);

  if(this->empty()) this->tail = new_node;
  else {
    new_node->set_next(this->head);
    this->head->set_prev(new_node);
  }
  this->head = new_node;
}
```





Discussion: Deleting or copying a list

Say we have a **linked-list** object x, consisting of the fields “head” and “tail”.

- The object runs out of scope and is getting deallocated. We did not define a destructor, so the default constructor is used.
  - What will this do? Is it what we want?
- We assign the value of x to another list object y, writing “y = x”. By default, this means that the property values “head” and “tail” are copied.
  - Is this reasonable? What could the user be expecting instead?

Or: We have a **singly-linked-list node** object, consisting of “item” and “next”.

- What will happen upon deallocation by default? What should happen?
- What will happen upon copying by default? What should happen?

Samanlikning: **list-template.zip** og **copying-and-moving.zip**.

Discussion: Resource efficiency

Usually we are not interested in the resource requirements of a single execution, but in understanding how the requirements behave as a function of a characteristic quantity, the **problem size *n***, that describes the magnitude of the task.

We distinguish between:

- **Time requirements**, describing the computing time. Where possible, this should be expressed in terms of actual CPU time (+ I/O time); the operating system will usually distribute CPU time between multiple processes.
- **Memory (or space) requirements**, describing the memory allocated to the program; depending on definition, this may include I/O size.
- **Worst-case performance**, which for any given problem size n corresponds to the input/special case of size n with the greatest requirements.
- **Average-case performance**, over many representative cases of size *n*.

Metrics closer to the hardware (*e.g.*, energy consumption) can also be relevant.

Discussion: Resource efficiency

Observations:

- **Performance analysis** is carried out by measurements; it is usually very hard to determine the worst case, therefore it is common to describe the **average-case performance**, *e.g.*, from random input.
- **Algorithm efficiency** can consider both the average and the worst case, but the average case usually requires a statistical analysis. Statements on the **worst case** can be very straightforward.
- There is no universal rule for how the **problem size *n*** should be defined. It is up to the person analysing an algorithm to define it appropriately. It should describe how complicated the task is.
- Distinguish between the efficiency of a program (or algorithm) and the complexity of the problem. The **complexity of the problem** is given by the **efficiency of the best possible program** (or algorithm).

Often we are interested in the **complexity class**, for example *linear time*, in terms of **asymptotic complexity** ($n \to \infty$) expressed in **Landau notation** *e.g.* as **O(*n*)**.

Overview: Sequential data structures

copying-and-moving.zip

- **Read/write** access to a data item at position k
  - For a dynamic array, $O(1)$ time; fast access by pointer arithmetics
  - For a singly linked list, $O(k)$ time, *i.e.*, $O(n)$ in the average/worst case
  - For a doubly linked list, $O(\min(k, n - k))$, which is still effectively $O(n)$

- **Iterating** over the data, *i.e.*, proceeding from one item to the next one
  - $O(1)$ both for dynamic arrays and for linked lists

- **Deleting** a data item at position k
  - For a dynamic array, $O(1)$ at the end, $O(n - k)$ in general
  - For a singly linked list, $O(1)$ at the head, or if we have a reference to the element at position k–1; otherwise, in general, $O(k)$
  - For a doubly linked list, $O(1)$ at the head or tail, or if we have a reference to that region of the list; in general, $O(\min(k, n - k))$

Remark: For linked lists, insertion/deletion as such takes constant time, once the node has been localized. However, getting to the node can take $O(n)$ time.

Overview: Sequential data structures

copying-and-moving.zip

- **Read/write** access to a data item at position k
  - For a dynamic array, $O(1)$ time; fast access by pointer arithmetics
  - For a singly linked list, $O(k)$ time, *i.e.*, $O(n)$ in the average/worst case
  - For a doubly linked list, $O(\min(k, n - k))$, which is still effectively $O(n)$

- **Iterating** over the data, *i.e.*, proceeding from one item to the next one
  - $O(1)$ both for dynamic arrays and for linked lists

- **Inserting** an additional data item at position k
  - For a dynamic array, $O(n)$ in the worst case, *i.e.*, whenever the capacity is exhausted; with free capacity, $O(1)$ at the end, $O(n - k)$ elsewhere
  - For a singly linked list, $O(1)$ at the head or tail, or if we have a reference to the element at position k–1; Otherwise, in general, $O(k)$
  - For a doubly linked list, $O(1)$ at the head or tail, or if we have a reference to that region of the list; in general, $O(\min(k, n - k))$

Remark: For linked lists, insertion/deletion as such takes constant time, once the node has been localized. However, getting to the node can take $O(n)$ time.

Stacks and queues

Example: **copying-and-moving.zip**

Queue

- nn, nb kø *m*.

Definition: A queue is a sequential (list-like) dynamic data structure that functions by the principle *first in, first out (FIFO)*.

- A queue class must provide a method for appending an element, usually called ***push*** and done on one end of the queue (*e.g.*, ***enqueue*** or ***push_back***), and a method for detaching an element, usually called ***pop*** and done at the other end of the queue (*e.g.*, ***dequeue*** or ***pop_front***).
- Singly and doubly linked lists are well suitable for implementing a queue, since both ***push*** and ***pop*** can be realized in constant time. However, a singly linked list should only be used if the data structure includes an explicit reference to the tail node; otherwise, the whole list needs to be traversed just to reach the tail, taking O(*n*) time.
- Dynamic arrays are less suitable for this purpose, requiring O(*n*) time for the ***push*** and ***pop*** operations in the long run (as their capacity gets exhausted).

– **Queues** function by the principle **"first in, first out" (FIFO)**
  - Can be implemented using a singly linked list (with a tail reference):
    – Attach (enqueue / push) new elements at the tail of the list only
    – Detach (dequeue / pop) elements from the head of the list only

Stacks and queues

Example: **copying-and-moving.zip**

- **Stacks** function by the principle **"last in, first out" (LIFO)**
  - Can be implemented using a singly linked list:
    - Attach (push) new elements at the head of the list only
    - Detach (pop) elements from the head of the list only
  - Can be implemented using a dynamic array:
    - Attach (push) new elements at the end of the array only
    - Detach (pop) elements from the end of the array only
- **Queues** function by the principle **"first in, first out" (FIFO)**
  - Can be implemented using a singly linked list (with a tail reference):
    - Attach (enqueue / push) new elements at the tail of the list only
    - Detach (dequeue / pop) elements from the head of the list only

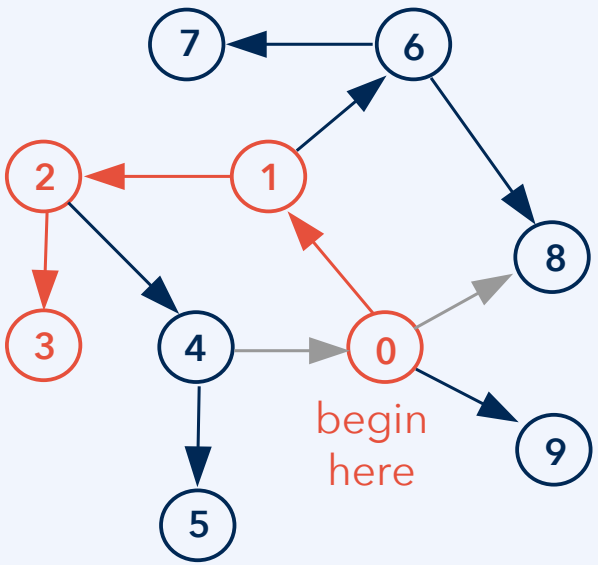
All these operations can be carried out in constant time;
in case of the push operation for the dynamic array, subject to capacity.

Graph traversal

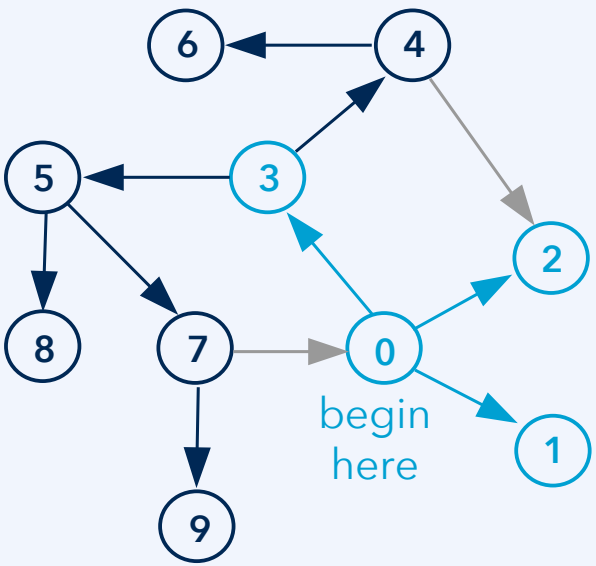
Traversal of graphs: Depth-first search and breadth-first search

DFS always proceeds from the most recently detected node (LIFO).
BFS always proceeds from the node that was detected earliest (FIFO).

depth-first search (DFS)



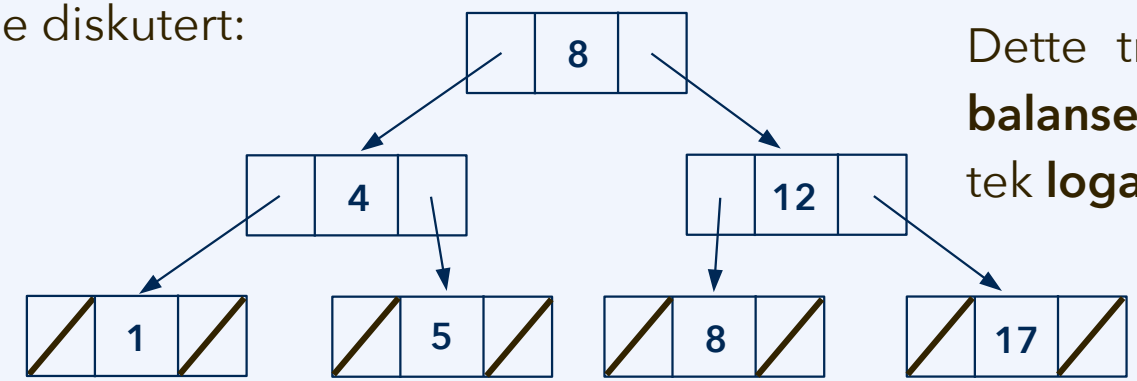
breadth-first search (BFS)



Note: Only elements *to which there is a path from the initial node* can be found.

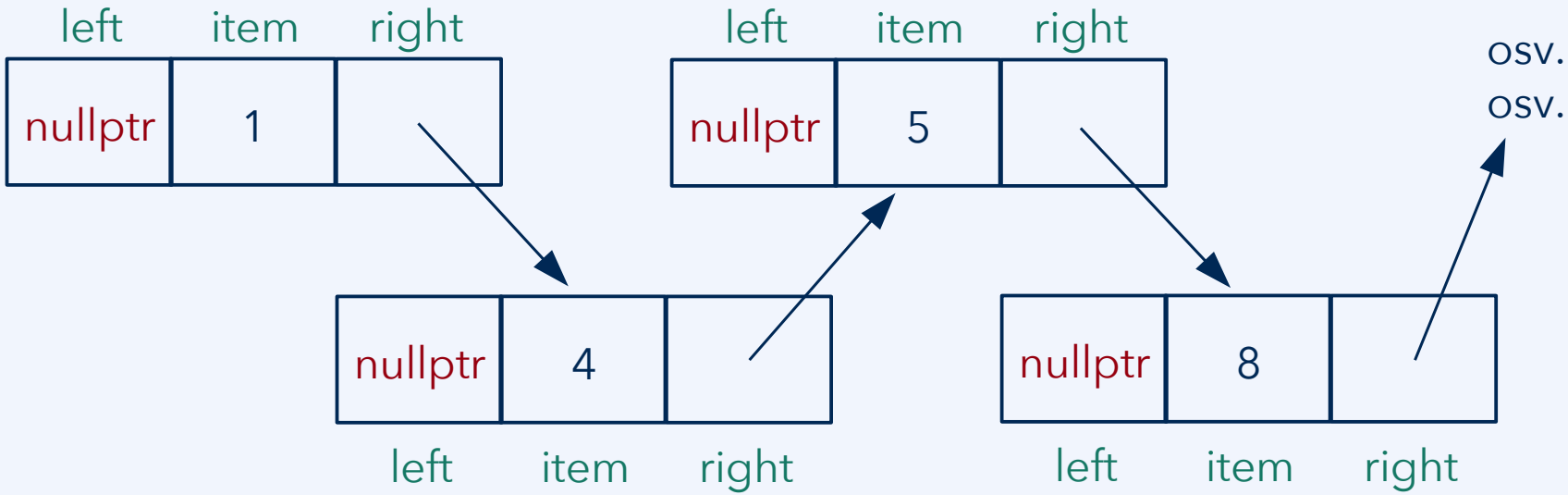
Tredatastrukturar: Binære søkjetre

Som allereie diskutert:



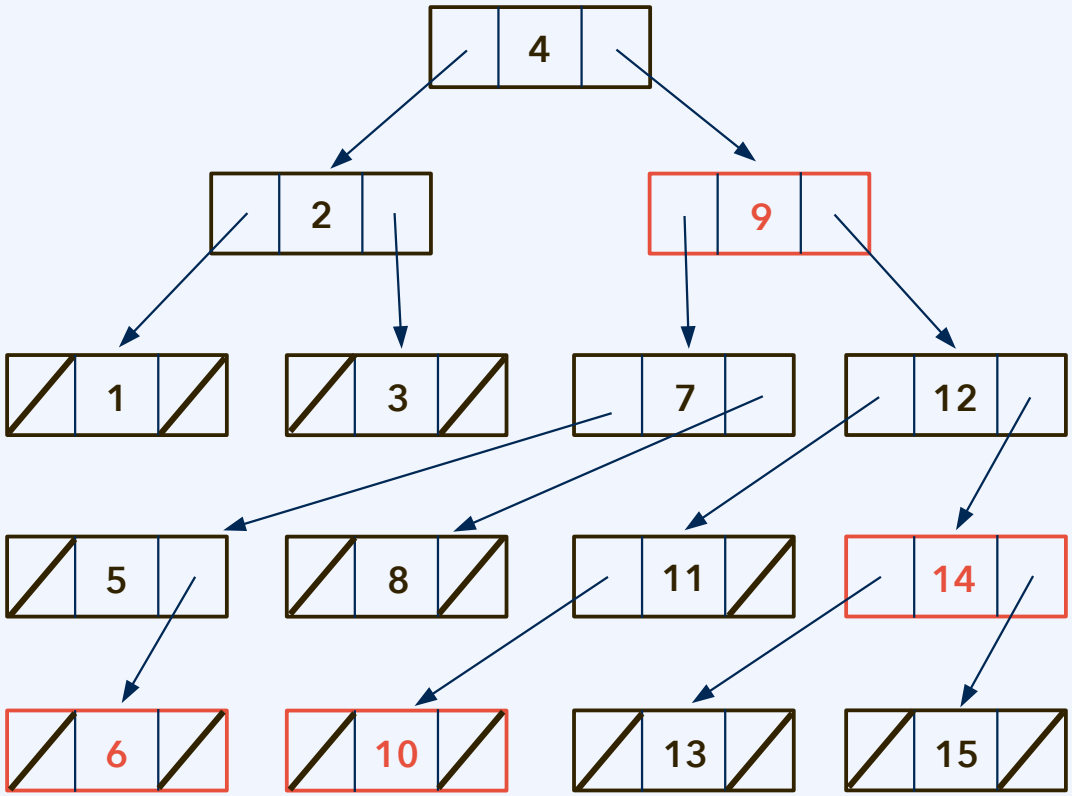
Dette treet er perfekt **balansert**. Operasjonar tek **logaritmisk tid**.

Kva var faren ved bruk av denne datastrukturen? Degenerasjon:



Tredatastrukturar: Raudt-svart-tre

Når treet er blitt altfor ubalansert, kan det vera lurt å ombyggje treet slik at det blir balansert igjen. **Raudt-svart-tre** er binære søkjetre med eit veldefinert kriterium på når vi skal intervenere i trestrukturen:



Reglar:

Nodar er anten **raude** eller **svarte**. (Ekstra bit eller bool.)

På alle stiar frå rota til blad eller halvblad (nodar som har eitt barn) er det like mange svarte nodar (**svarthøgda**).

Raude nodar har **berre svarte born**.

Glossar

Breidd-fyrst-søk (BFS)

Kø

Stakk [datastruktur]

Glossar

Breidd-fyrst-søk (BFS)

Definisjon: graftraversering som besøkjer nodane etter FIFO-regelen («first in, first out»), dvs. i den same rekkjefylgja som dei blir oppdaga

Kø

Definisjon: dynamisk sekvensiell datastruktur som fylgjer FIFO-regelen («first in, first out»)

Stakk [datastruktur]

Definisjon: dynamisk sekvensiell datastruktur som fylgjer LIFO-regelen («last in, first out»)

Noregs miljø- og biovitskaplege universitet

Institutt for datavitskap

Digitalisering på Ås

Problemliste D

INF205

5. mars 2026

Noregs miljø- og biovitskaplege universitet

Institutt for datavitskap

Digitalisering på Ås

Spørjetime oppgåve 2

INF205

5. mars 2026

Noregs miljø- og biovitskaplege universitet

Institutt for datavitskap

Digitalisering på Ås

INF205
Ressurseffektiv programmering

3 Datastrukturar

3.1 Grafdatastrukturar

3.2 Kode-data-samutvikling

3.3 Listedatastrukturar